

从零手写水体渲染器 · 完整学习方案

本方案目标：让一个**图形学初学者**，在不依赖 AI 改代码、不靠调参蒙混的前提下，通过**自己一行行敲完**，最终独立写出本仓库 WaterSurfaceRendering 的全部代码，并真正理解 **Vulkan 管线** → **物理波浪(FFT)** → **网格** → **Shader 渲染** 的完整原理。

铁律（请严格遵守，否则学不到东西）

1. 每个阶段的代码**必须自己手敲**，不准复制本仓库源码，不准让 AI 代写。
2. 遇到不懂先查规范/书，再动手；卡住超过 2 小时可看仓库对应文件**找思路**，但看完要**关掉源码凭记忆重写**。
3. 每阶段都有"验收标准"，没达到不许进下一阶段。
4. 仓库源码只当"标准答案"用于**对照检查**，不当"抄写模板"。
5. 调参是最后才做的事；学习期一律用固定参数，先把"为什么这么写"搞懂。

学习路线总览

阶段0 准备：环境 + 数学 + C++/Vulkan 心智模型
|
阶段1 Vulkan 裸API 画三角形（不封装，单文件，吃透每个对象）
|
阶段2 把裸代码重构成封装层（对照 src/vulkan/ 的类划分）
|
阶段3 Application 框架 + 窗口 + 主循环 + 相机 (src/core, scene/Camera)
|
阶段4 纹理 + 描述符 + UB0（让 shader 能读数据）
|
阶段5 ★Tessendorf 海洋谱 + FFT 物理 (src/scene/WSTessendorf) — 纯 CPU，可脱离 Vulkan 单测
|
阶段6 网格生成 + CPU→GPU 上传 + 顶点位移 shader (src/scene/WaterSurfaceMesh + .vert)
|
阶段7 ★片元着色：Fresnel/折射/水体光学 (WaterSurfaceMesh.frag)
|
阶段8 Preetham 天空模型 + 合成 (src/scene/SkyModel, SkyPreetham, .frag)
|
阶段9 GUI(ImGui) + Profiling + 资源回收 + 收尾打磨

预计总时长：~~全职 610 周 / 业余 35 个月~~。这是合理的，Vulkan 本身就陡。

阶段 0 | 准备（约 3~5 天）

0.1 必须先具备的前置

- **C++17**：智能指针(`unique_ptr`)、RAII、移动语义、模板基础、`std::vector` / `std::array`。
 - 自测：能解释为什么本项目大量用 `std::unique_ptr` 管理 Vulkan 句柄，析构顺序为何重要（见 `Application.h:132` 注释 "Order of declaration is important!"）。
- **线性代数**：向量点乘/叉乘、矩阵、齐次坐标、MVP（Model/View/Projection）变换、法线变换。
- **图形学基础**：光栅化管线阶段、NDC、深度测试、重心插值。
- **复数与傅里叶变换的"直觉"**：欧拉公式 $e^{i\theta} = \cos\theta + i\sin\theta$ 、频域↔空间域、IFFT 概念。
（阶段 5 会深入，这里只要不怕复数即可。）

学习资源建议：[vulkan-tutorial.com]、《Real-Time Rendering》相关章、Tessendorf 《Simulating Ocean Water》原文。

0.2 环境搭建（自己装，不要让 AI 代劳）

- 安装 Vulkan SDK（含 validation layers、`glslc / shaderc`）。
- 装 CMake、C++17 编译器。
- 准备依赖库：GLFW3（窗口）、GLM（数学）、SPDLOG（日志）、FFTW 3.3.10（FFT）、Dear ImGui、stb。
- 写一个最小 `CMakeLists.txt`，能链接 Vulkan + GLFW + GLM，编译出一个空 `main()`。
- **开启 Validation Layers**：这是你整个学习期最重要的老师，必须从第一天就打开。

阶段验收：空程序能编译运行，validation layer 在控制台有输出，`vkEnumerateInstanceExtensionProperties` 能打印出扩展列表。

阶段 1 | Vulkan 裸 API 画三角形（约 1.5~2 周）★最难也最关键

心法：先在一个 `.cpp` 文件里把 1000+ 行的三角形流程裸写一遍，不要封装。
目的是把 Vulkan 每个对象的"为什么存在、和谁交互"刻进脑子。封装是阶段 2 的事。

1.1 必须亲手实现的对象（按依赖顺序）

对照仓库 `src/vulkan/` 的文件名，理解每个对象职责：

顺序	对象	你要搞懂的问题	对照文件
1	Instance + Debug Messenger	校验层怎么挂？扩展怎么查？	<code>Instance.cpp</code>
2	Surface	窗口系统和 Vulkan 如何桥接？	<code>Surface.cpp</code>
3	PhysicalDevice	怎么挑显卡？队列族(queue family)是什么？	<code>PhysicalDevice.cpp</code>
4	Logical Device + Queues	逻辑设备 vs 物理设备区别？队列怎么取？	<code>Device.cpp</code>

5	SwapChain	交换链/呈现模式/图像数量/extent 选择	SwapChain.cpp
6	ImageView	为什么 image 要套一层 view?	ImageView.cpp
7	RenderPass	attachment/subpass/dependency 三者关系	RenderPass.cpp
8	ShaderModule	SPIR-V 怎么来的? GLSL 怎么编译?	ShaderModule.cpp
9	Pipeline	图形管线那十几个 state 各管什么?	Pipeline.cpp
10	Framebuffer	它把 renderpass 和实际 imageview 绑起来	SwapChain::CreateFramebuffers
11	CommandPool + CommandBuffer	录制 vs 提交, one-time submit	CommandPool/CommandBuffer.cpp
12	同步: Semaphore + Fence	GPU-GPU vs CPU-GPU 同步差异	SwapChain::CreateSyncObjects

1.2 渲染循环（自己写）

实现: `acquireNextImage` → 录制命令 → 提交(等待/触发信号量) → `present`。

理解仓库 `SwapChain::DrawFrame / PresentFrame` (`SwapChain.cpp:213`) 的信号量链。

1.3 必须能口头回答（写在笔记里）

- 一帧从 CPU 到屏幕, 经历了哪些 Vulkan 调用?
- 为什么需要两个 semaphore (`imageAvailable / renderComplete`) 和一个 fence?
- `RenderPass` 的 `subpass dependency` 解决了什么竞争?
- `Pipeline` 为什么是"不可变(`immutable`)"的, 改个视口要重建吗?

阶段验收:

- 屏幕出现一个**彩色三角形**, 窗口可缩放 (处理 `swapchain` 重建)。
- `validation layer` **零报错**。
- 关掉所有源码, 能在白纸上画出"从 `Instance` 到 `Present`"的对象依赖图。

常见坑：image layout 没转换、framebuffer extent 与 swapchain 不一致、销毁顺序错误导致 validation 报 "object in use"。

阶段 2 | 重构成封装层（约 1 周）

把阶段 1 的裸代码，自己重构成和仓库一致的类划分（不要抄实现，只借鉴接口设计）：

- `vkp::Instance / Surface / PhysicalDevice / Device / SwapChain`
- `vkp::RenderPass / Pipeline / ShaderModule`
- `vkp::CommandPool / CommandBuffer`
- `vkp::Buffer`（封装 buffer + memory + map/flush，见 `Buffer.cpp`）

关键设计点（理解后自己实现）

- **RAII**：构造即创建，析构即销毁；用 `operator VkXxx()` 隐式转换句柄（仓库到处用这招）。
- **Buffer 抽象**：`Create / Map / CopyToMapped / FlushMappedRange / StageCopy`，搞懂 **host-visible vs device-local**、**non-coherent memory** 为何要 **flush**（`Device::GetNonCoherentAtomSizeAlignment`）。
- **队列抽象**：理解仓库为何把 `graphics` 和 `transfer` 用同一个队列族（`Application.h:144` 注释）。

阶段验收：用你自己的封装类重新画出三角形，代码组织接近 `src/vulkan/`，且能讲清每个类的单一职责。

阶段 3 | Application 框架 + 窗口 + 主循环 + 相机（约 4~6 天）

3.1 Application 基类（对照 `core/Application.h`）

自己实现一个抽象基类，暴露三个虚函数：

```
virtual void Start() = 0;           // 主循环前一次性初始化
virtual void Update(Timestep dt) = 0; // 每帧逻辑
virtual void Render(...) = 0;       // 每帧录制+提交
```

- 实现 `Run() = Init() + Loop()`。
- `Init()` 里按序：`SetupWindow` → `SetupVulkan` → `Start`。

- `Loop()`: `glfwPollEvents` → 处理 `resize` → 计算 `dt` → `Update` → `AcquireNextImage` → `Render` → `DrawFrame` → `PresentFrame` (对照 `Application.cpp:47-114`)。

3.2 Window (GLFW 封装, 对照 `core/Window.cpp`)

- 创建窗口、注册回调 (`resize/mouse/key`), 把事件转发给 `Application` 的虚函数 (`OnFramebufferResize/OnMouseMove/OnKeyPressed`)。

3.3 工具设施

- `Timestep`、`Timer` (帧时间)、`Log` (包 `spdlog`)、`Assert`、`Profile` (先放空实现, 阶段 9 再补)。
- 实现 `BeginOneTimeCommands` / `EndOneTimeCommands` (一次性传输命令, 后续上传纹理/网格要用)。

3.4 FPS 相机 (对照 `scene/Camera.cpp`)

- 自己实现: 位置/pitch/yaw/fov → `view` 矩阵; `perspective` → `proj` 矩阵。
- 键鼠控制: WASD 移动、鼠标转视角。
- 注意 Vulkan 的 Y 翻转: `proj[1][1] *= -1` (见 `WaterSurfaceMesh.cpp:171`)。

阶段验收: 做一个 `ClearColorApp : public Application`, 能用相机在场景里"飞", 窗口每帧清成某颜色, `resize` 不崩。

阶段 4 | 纹理 + 描述符 + UBO (约 1 周)

这一阶段让 `shader` 能"读到数据", 是水体渲染的前置基建。

4.1 Uniform Buffer (UBO)

- 用阶段 2 的 `Buffer` 建一个 `host-visible` 的 UBO, 每帧 `CopyToMapped` 一个 MVP 矩阵。
- 理解 **uniform buffer offset alignment** (`Device::GetUniformBufferAlignment`): 为什么两个 UBO 塞进同一 `buffer` 时要对齐 (`WaterSurfaceMesh.cpp:246` 的双 UBO 布局)。

4.2 描述符系统 (对照 `vulkan/Descriptors.cpp`)

自己实现三件套:

- `DescriptorSetLayout::Builder` (`AddBinding` 链式) —— 描述 `shader` 要哪些资源。
- `DescriptorPool` (`AllocateDescriptorSet`)。
- `DescriptorWriter` (`AddBufferDescriptor` / `AddImageDescriptor` / `UpdateSet`)。

- 搞懂 **set / binding / descriptor type** 的概念，对照水体的 4 个 binding
(`WaterSurfaceMesh.cpp:337 CreateDescriptorSetLayout`):
binding0=VertexUBO, binding1=WaterSurfaceUBO, binding2=位移图, binding3=法线图。

4.3 Texture2D (对照 `vulkan/Texture2D.cpp` + `Image.cpp` + `ImageView.cpp` + `Sampler.cpp`)

- 实现: 创建 image → 分配内存 → 创建 view → 创建 sampler。
- 核心难点 image layout 转换**: `UNDEFINED` → `TRANSFER_DST` → `SHADER_READ_ONLY`
(`Image::TransitionLayout_*`), 用 pipeline barrier。
- 实现 `CopyFromBuffer` (staging buffer → image, `Texture2D::CopyBufferToImage`)。
- 先用 stb 加载一张普通图片贴到一个 quad 上验证 (做个 textured quad demo)。

4.4 练手 demo

写一个"带纹理的旋转四边形": 顶点缓冲 + 索引缓冲 + UBO(MVP) + 一张贴图 + 描述符集, 跑通。

阶段验收: 贴图四边形正确显示并随 MVP 旋转; 理解 staging buffer → device 的完整传输链; validation 零报错。

阶段 5 | ★Tessendorf 海洋谱 + FFT 物理 (约 1.5~2 周) 核心物理

这一阶段完全脱离 Vulkan, 在纯 C++ 控制台程序里做, 把结果输出成图片或文本验证。

对照 `src/scene/WSTessendorf.cpp/.h`, 但先读论文再写代码。

5.1 必读理论 (Tessendorf 1999 《Simulating Ocean Water》)

理解这条主线:

- 海面高度场是大量正弦波叠加 → 频域里就是一组复振幅 $h(k)$ 。
- Phillips 谱**: 风驱动波的能量分布, 决定每个波矢量 k 的初始振幅
(对照 `WSTessendorf.h:249 PhillipsSpectrum`, 搞懂公式里每一项: 风速 L 、风向 $\cos$ 因子、阻尼 damping)。
- 初始频域振幅** $h_0(k)$: 高斯随机数 $\times \sqrt{(\text{Phillips})}$ (`WSTessendorf.h:237 BaseWaveHeightFT`)。
- 色散关系** $\omega(k)=\sqrt{gk}$: 决定波随时间如何相位演化
(`DispersionDeepWaves`, 以及为什么要量化 `QDispersion` 让动画循环)。
- 时间演化** $h(k,t) = h_0(k)e^{i\omega t} + h_0^*(-k)e^{-i\omega t}$ (`WSTessendorf.h:265 WaveHeightFT`) —— 保证实数高度场。

6. **IFFT**: 把频域 $h(k, t)$ 逆变换回空间域 $\rightarrow$ 得到每个网格点的真实高度。
7. **choppy (横向位移)**: 用 $ik/|k|$ 加权得到水平位移, 让浪尖更尖 (Lambda 参数)。
8. **法线/斜率**: 频域里乘 ik 得到导数, IFFT 后得到斜率, 组装法线。

5.2 必须亲手实现 (建议先用朴素 DFT 跑通, 再换 FFTW 加速)

按 `WSTessendorf` 的方法名, 自己实现:

- `ComputeWaveVectors()`: 生成 $N \times N$ 的离散波矢量 (`WSTessendorf.cpp:60`, 搞懂 $2\pi n/L$ 那条公式)。
- `ComputeGaussRandomArray()`: 高斯随机数组。
- `ComputeBaseWaveHeightField()`: 算 h_0 、 h_0^* 、色散 ω (`WSTessendorf.cpp:105`)。
- `ComputeWaves(t)`: 每帧——
 1. 算当前频域高度场;
 2. 频域乘 ik 得斜率/位移/位移导数;
 3. 执行 IFFT (这就是 FFT 所在! `WSTessendorf.cpp:284`, IFFT 调用在 342-378);
 4. 处理符号翻转 $(-1)^{(m+n)}$ (频谱中心化, `kSigns`);
 5. 组装 `m_Displacements` (vec4: `dispX, height, dispZ, jacobian`) 和 `m_Normals` (vec4: `slopeX, slopeZ, dDxdx, dDzdz`);
 6. `NormalizeHeights` 归一化高度到 $[-1, 1]$ 。

5.3 FFT 具体怎么接

- 先自己写一个朴素 2D DFT ($O(N^4)$, 慢但能验证正确性), 跑 $N=32$ 。
- 再换 FFTW: `fftwf_plan_dft_2d(..., FFTW_BACKWARD, FFTW_MEASURE)` (`SetupFFTW WSTessendorf.cpp:150`), `fftwf_execute` 执行 (`ComputeWaves` 内)。
- 理解: `plan` 是"预先规划好的变换路径", 建一次反复执行; `BACKWARD` = 逆变换 (频域 $\rightarrow$ 空间)。
- 可选: 用 OpenMP `#pragma omp sections` 并行多个 FFT (`WSTessendorf.cpp:338`)。

5.4 验证手段 (不靠眼睛, 靠输出)

- 把某一帧的高度场写成灰度 PNG, 看是否像水面 (有连续起伏、无突刺)。
- 检查归一化后高度在 $[-1, 1]$ 。
- 改风速/风向, 看波纹方向是否合理 (先理解, 别急着调参玩)。

阶段验收:

- 控制台程序能稳定输出每帧的位移图/法线图数据。
- 能口述"从 Phillips 谱到屏幕高度"的完整数据流。
- 关掉源码, 能独立写出 `ComputeWaves` 的主干逻辑。

常见坑：频谱中心化符号翻转漏了（水面会错位/抖动）、复数共轭对称没保证导致高度出现虚部、波矢量 $k=0$ 没特判（除零）。

阶段 6 | 网格生成 + CPU→GPU 上传 + 顶点位移（约 1 周）

把阶段 5 的物理数据，接到阶段 1-4 的 Vulkan 管线上。对照 `scene/WaterSurfaceMesh.cpp` + `shaders/WaterSurfaceMesh.vert`。

6.1 网格生成（CPU，自己写）

- `CreateGridVertices(N, scale)`：生成 $(N+1) \times (N+1)$ 个顶点， $y=0$ 的平面网格，带 UV（`WaterSurfaceMesh.cpp:500`）。
- `CreateGridIndices(N)`：每个 quad 拆两个三角形（`WaterSurfaceMesh.cpp:538`）。
- 实现 `Mesh<Vertex>`（顶点/索引缓冲 + `UpdateDeviceBuffers` + `Render`），对照 `scene/Mesh.h`。

6.2 把位移图/法线图做成 Vulkan 纹理

- 用阶段 5 的输出，每帧拷进 staging buffer，再 `CopyFromBuffer` 到两张 `R32G32B32A32_SFLOAT` 纹理（`WaterSurfaceMesh.cpp:642 UpdateFrameMaps`）。
- 为每个分辨率(16~1024)预建一对纹理（`CreateFrameMaps :596`）—— 理解为什么预建（切分辨率不重分配）。
- 理解 staging buffer 的内存布局：顶点 + 索引 + 两张 map 拼在一块大 buffer 里（`CreateStagingBuffer :571`，注意对齐 `AlignSizeTo`）。

6.3 顶点着色器（自己写 GLSL，对照 `WaterSurfaceMesh.vert`）

核心 6 行：

- 采样位移图 → `outPos = inPos + D.xyz`，高度乘振幅 `WSHeightAmp`。
- `gl_Position = proj * view * model * pos`。
- 采样法线图斜率 → 用 choppy 校正算世界法线（`vert:34`，搞懂为什么是 `-slope/(1+choppy*dDxdx)`）。

6.4 每帧更新（对照 `Update :145` 和 `PrepareRender :156`）

- `Update`：推进时间 → `ComputeWaves` → 拷数据到 staging。
- `PrepareRender`：填 UBO（MVP/振幅/choppy）、更新描述符、`UpdateFrameMaps` 上传纹理。

- 理解纹理上传与顶点着色器读取的同步 (pipeline stage: `VK_PIPELINE_STAGE_VERTEX_SHADER_BIT`)。

阶段验收：屏幕出现一片起伏波动的网格水面（先用线框模式 `fillModeNonSolid` 看几何，确认浪在动）。validation 零报错。

常见坑：UBO 双缓冲对齐错（画面错乱）、纹理 layout 在上传/采样间没正确转换、描述符集没标 dirty 导致绑了旧纹理。

阶段 7 | ★片元着色：Fresnel / 折射 / 水体光学（约 1.5 周）核心渲染

对照 `shaders/WaterSurfaceMesh.frag` + `WaterSurfaceUBO` (`WaterSurfaceMesh.h:260`)。先理解光学原理再写 shader。

7.1 理论 (Tessendorf [1] + Baboud&Décoret [3] + Premoze&Ashikhmin [2])

最终出射辐射度：

$$L = F_r \cdot L_s + F_r \cdot L_a + F_t \cdot L_u$$

逐项搞懂：

- **F_r / F_t** ：Fresnel 反射率/透射率，空气↔水 IOR=1.0/1.33477 (`frag:40`)。自己推 Fresnel 公式或用 Schlick 近似。
- **L_s** ：太阳直接反射进相机 (Blinn-Phong 高光)。
- **L_a** ：天空漫反射光 (阶段 8 的 Preetham 提供)。
- **L_u** ：水下上来、被折射进相机的光。

7.2 水下光线追踪 (Baboud&Décoret 算法，自己实现)

对照 README "How it Works" 第 1-9 步：

1. 相机→片元构造视线 `r_v`。
2. 算 Fresnel 反射率。
3. 折射方向 `t` (Snell 定律)，构造折射射线 `r_t`。
4. `r_t` 与水下深度 `d` 处的水平假想地面求交 `p_g` (`frag:46 kTerrainBoundPlane`)。
5. 采样地面颜色/法线 (可用 value noise 做简单海底)。

6. 沿 `r_t` 路径做吸收 + 散射衰减得 `L_u`。

7.3 水体光学系数（理解，别死记）

- 吸收系数 / 散射系数 / 后向散射系数：来自 Jerlov 水体分类（`WaterSurfaceMesh.h:293` 起的 `s_kWaterTypesCoeffs`）。
- RGB 三波长 (680/550/440nm) 分别衰减 → 这就是水为什么偏蓝绿。
- 实现 `ComputeScatteringCoefPA01` / `ComputeBackscatteringCoefPA01`（`WaterSurfaceMesh.h:369`）。

7.4 写 frag shader

按上面把 `L` 各项组合，先实现"只有反射"，再加折射，再加水下衰减，**分步验证**每加一项画面合理。

阶段验收：水面有真实的"近处透绿、远处反天/反光、掠射角强反射(Fresnel)"效果。能逐项解释画面里每块颜色来自公式哪一项。

常见坑：Fresnel 用错入射角、折射射线与平面求交方向反了（采样到水面上方）、RGB 衰减只用单通道导致水不变蓝绿。

阶段 8 | Preetham 天空模型 + 合成（约 4~6 天）

对照 `scene/SkyModel.cpp`、`scene/SkyPreetham.cpp`、`shaders/SkyPreetham.frag`、`shaders/FullScreenQuad.vert`。

8.1 Preetham 解析天空（[5] Preetham et al. 1999）

- 理解天空亮度是 太阳方向、观察方向、浊度(turbidity) 的解析函数。
- 实现 Perez 模型的 A~E 系数、天顶亮度 `ZenithLum`（`SkyPreetham.cpp` 计算，传进 `WaterSurfaceUBO.sky`）。
- 自己实现 `SkyModel` 类：算系数 → 填 UBO → 提供给水体 frag 当 `L_a`。

8.2 天空渲染（全屏）

- `FullScreenQuad.vert`：用一个覆盖全屏的三角形/四边形，无需顶点缓冲（`gl_VertexIndex` 生成）。
- `SkyPreetham.frag`：按视线方向算天空色。
- **绘制顺序：**先画天空（不写深度），再画水面，再画 GUI（`WaterSurface.cpp:183`）。理解为什么天空不写深度。

8.3 整合

- 水体 frag 的 `L_s` (太阳)、`L_a` (天空) 现在都由 `SkyModel` 提供, 画面统一。

阶段验收: 能切换日出/正午/黄昏 (改太阳方向), 天空与水面光照一致联动。理解 `SkyLuminance` 如何同时服务天空和水面。

阶段 9 | GUI + Profiling + 资源回收 + 收尾 (约 4~6 天)

9.1 Dear ImGui 接入

- 自己接 ImGui 的 Vulkan + GLFW backend (对照 `Gui.cpp`)。
- 实现参数面板: 分辨率、风速/风向、Phillips 振幅、choppy、动画速度、水体类型、光照 (对照 `WaterSurfaceMesh.cpp:796 ShowWaterSurfaceSettings / :906 ShowMeshSettings`)。
- **此时才允许调参:** 因为你已经懂每个参数的物理含义了。
- 理解 GUI 状态切换: `ESC` 切 GUI/相机控制 (`WaterSurface.h:91 States`)。

9.2 Profiling

- 实现 `Profile` (`core/Profile.cpp`): `VKP_PROFILE_SCOPE` 计时, 做个 profiling 窗口 (README 的 `profiling-window`)。

9.3 健壮性收尾

- `swapchain` 重建时所有资源正确重建 (`OnFramebufferResize :96`)。
- 切分辨率时 `m_FrameMapNeedsUpdate`、描述符 `dirty` 逻辑正确。
- `shader` 热重载 (`F1, RecompileShaders`)。
- 全程 `validation layer` 零报错、关闭时无内存/句柄泄漏。

最终验收 (项目完成标准):

- 不看仓库源码, 能从空目录开始, 独立写出可运行的完整水体渲染器。
 - 能讲清: 一帧里 CPU 算物理 → 上传纹理 → 顶点位移 → 片元光学 → 天空合成 → 呈现的全链路。
 - 能回答本文每阶段的自测题。
-

通用学习方法 (每个阶段都这么做)

1. **读 → 想 → 写 → 验 → 复盘:** 先读理论/规范, 想清楚数据流, 自己写, 跑验收, 再对照仓库复盘差异。
2. **每阶段写一份笔记:** 用自己的话画数据流图 + 对象依赖图 + 踩坑记录。

- 3. **Validation Layer 当老师**：任何报错都先读懂再改，不要瞎试。
- 4. **小步验证**：复杂 shader/物理分步骤加，每加一步确认画面/数据合理。
- 5. **"关源码重写"训练**：看了仓库找思路后，必须关掉源码凭理解重写，否则等于抄。
- 6. **禁止过早调参**：参数含义没吃透前一律用默认值，把"为什么"搞懂优先于"好不好看"。

关键文件索引（对照检查用，非抄写用）

模块	核心文件
入口/主循环	src/main.cpp、src/core/Application.cpp/.h
Vulkan封装	src/vulkan/* (Instance/Device/SwapChain/Pipeline/RenderPass/Buffer/Texture2D/Descriptors...)
相机/窗口	src/scene/Camera.cpp、src/core/Window.cpp
★物理FFT	src/scene/WSTessendorf.cpp/.h
★网格/纹理/上传	src/scene/WaterSurfaceMesh.cpp/.h
★顶点着色	src/shaders/WaterSurfaceMesh.vert
★片元着色	src/shaders/WaterSurfaceMesh.frag
天空	src/scene/SkyModel/SkyPreetham.cpp、src/shaders/SkyPreetham.frag、FullscreenQuad.vert
应用装配	src/WaterSurface.cpp/.h
GUI/性能	src/Gui.cpp、src/core/Profile.cpp

提醒：本项目**没有** LOD / 四叉树 / 曲面细分。网格是单块等密度 grid，分辨率手动选。
学完本方案后，可把 README 的 TODO（Endless 平铺、Gerstner 波、compute shader FFT、Foam）作为进阶自我挑战。